

RetailMart Enterprise Sales Data Platform

Product-Company Data Engineering Portfolio Project - Project 1

PostgreSQL + Python + ETL + Google Cloud Storage + BigQuery

This document is a real learning assignment. It contains business context, exact tasks, deliverables, acceptance criteria, datasets, and review questions. It does not claim that you completed the work. You must implement and understand it yourself.

1. Project Outcome

You will build a small but production-minded batch data platform for a fictional Indian retail company. The finished solution will ingest seven daily transaction files and multiple master-data sources, validate and transform records, load operational data into PostgreSQL, archive curated files in Google Cloud Storage, and publish analytics tables in BigQuery.

- A normalized PostgreSQL operational layer
- A modular Python ETL pipeline
- Rejected-record handling with reason codes
- Pipeline audit and reconciliation metrics
- GCS raw/curated storage layout
- BigQuery star schema with partitioning and clustering
- Twenty business analytics queries
- Tests, documentation, architecture diagram, and GitHub-ready repository

2. Business Scenario

RetailMart operates 30 stores in multiple Indian regions. Each store sends a daily sales CSV after closing. Customer, product, store, promotion, and inventory master data are maintained separately. Finance and category teams currently combine files manually, leading to late reports, duplicate transactions, invalid identifiers, and inconsistent revenue figures.

Management wants daily reporting available by 8:00 AM. The first release is a batch pipeline. Streaming, Airflow, Spark, and dashboards are explicitly deferred so that you first master Python, PostgreSQL, ETL design, GCS, and BigQuery.

2.1 Stakeholders

Stakeholder	Need
Finance	Trusted net-sales and discount figures
Category Management	Product/category performance
Store Operations	Store and inventory analysis
Marketing	Customer and promotion insights
Data Engineering	Reliable, restartable, auditable pipeline
Analytics	Clean BigQuery tables

2.2 Success Metrics

- All supplied files are detected and processed once.
- Invalid rows never enter curated tables.
- Every rejected row contains one or more reason codes.
- PostgreSQL and BigQuery reconciliation differences are zero for accepted rows.
- A rerun does not duplicate previously loaded transactions.
- The pipeline produces a run-level audit record and readable logs.

3. Scope

3.1 In Scope

- CSV and JSON source ingestion
- Schema and business-rule validation
- Full loads for master data
- Incremental daily loads for sales
- PostgreSQL staging, core, mart, and audit schemas
- Python modules, configuration, logging, exception handling
- GCS raw and curated zones
- BigQuery dimensions and sales fact table
- Partitioning, clustering, MERGE/upsert logic
- Data quality, reconciliation, unit tests, documentation

3.2 Out of Scope

- Claiming this as company employment experience
- Real customer or payment data
- Kafka or real-time streaming
- Airflow/Composer scheduling
- Spark/PySpark
- BI dashboard creation
- Production secrets in GitHub

4. Source Data

File/Source	Format	Load Type	Purpose
customers.csv	CSV	Full	Customer master
products.csv	CSV	Full	Product master
stores.csv	CSV	Full	Store master
promotions.csv	CSV	Full	Promotion rules
inventory_snapshot.csv	CSV	Snapshot	Stock levels
sales_YYYY-MM-DD.csv	CSV	Incremental	Daily transactions
holiday_calendar.json	JSON	Full	Holiday enrichment
exchange_rates.json	JSON	Snapshot	Optional currency enrichment

The supplied raw data intentionally includes defects. Do not manually edit raw files. Your code must identify valid and invalid records.

5. Target Architecture

Raw CSV/JSON files -> Python extraction and schema checks -> validation and rejected records -> PostgreSQL staging -> PostgreSQL core and mart transformations -> curated Parquet/CSV -> GCS -> BigQuery dimensions/fact -> analytics SQL.

5.1 Required GCS Layout

gs://<bucket>/retailmart/raw/source=<name>/load_date=YYYY-MM-DD/...

gs://<bucket>/retailmart/curated/table=<name>/business_date=YYYY-MM-DD/...

gs://<bucket>/retailmart/rejected/source=<name>/load_date=YYYY-MM-DD/...

6. Data Model Requirements

6.1 PostgreSQL

- staging: raw-shaped temporary/load tables
- core: customers, products, stores, promotions, sales, inventory_snapshot
- mart: daily_sales_summary, product_performance, customer_value
- audit: pipeline_run, file_audit, rejection_summary

6.2 BigQuery

- dim_customer, dim_product, dim_store, dim_date
- fact_sales partitioned by business_date
- Cluster fact_sales by store_id and product_id
- Use MERGE or load-job controls to prevent duplicate sale_id values

7. Mandatory Data Quality Rules

Rule ID	Rule
DQ-001	Primary identifiers must not be null or blank.
DQ-002	sale_id and master identifiers must be unique within their entity.
DQ-003	quantity must be greater than zero.
DQ-004	unit_price and product list_price must be non-negative.
DQ-005	discount_pct must be between 0 and 50 inclusive.
DQ-006	business_date must match the date in the daily filename.
DQ-007	event_ts must parse and must not precede business_date.
DQ-008	store_id, customer_id, and product_id must exist in accepted master data.
DQ-009	payment_method must be UPI, CARD, CASH, or WALLET.
DQ-010	net_amount = quantity * unit_price * (1 - discount_pct/100), rounded to 2 decimals.
DQ-011	Raw files must remain unchanged.
DQ-012	Rerunning a processed file must not duplicate accepted sales.

8. Functional Requirements

ID	Requirement
FR-001	Discover all expected files and fail clearly when mandatory files are missing.
FR-002	Read CSV and JSON using reusable extraction functions.
FR-003	Validate required columns before row processing.
FR-004	Standardize column names and data types.
FR-005	Trim text and normalize categorical values.
FR-006	Apply all DQ rules and attach reason codes.
FR-007	Write rejected rows without losing original source values.
FR-008	Load accepted master data to PostgreSQL using upsert logic.
FR-009	Load sales incrementally and idempotently.
FR-010	Create gross_amount, discount_amount, net_amount, and margin_amount.
FR-011	Write audit records with source, counts, status, timestamps, and error message.
FR-012	Reconcile source, accepted, rejected, PostgreSQL, and BigQuery counts.
FR-013	Upload raw/curated/rejected artifacts to GCS.
FR-014	Load curated tables to BigQuery.
FR-015	Create analytics tables/views and answer business questions.
FR-016	Exit with non-zero status when the run fails.
FR-017	Use environment variables; do not hard-code passwords or keys.
FR-018	Generate logs with run_id and file name.
FR-019	Unit-test core validation and transformation functions.
FR-020	Document setup, execution, assumptions, and evidence in README.

9. Non-Functional Requirements

Area	Requirement
Correctness	No known invalid record in curated tables
Idempotency	Same input can be rerun safely
Observability	Logs and audit tables explain every run

Maintainability	Modules have single responsibilities
Security	Secrets excluded from source control
Performance	Use vectorized pandas and batched database writes
Scalability	Design supports replacing pandas with Spark later
Portability	Works on macOS with documented commands
Cost	Use GCP free-tier-conscious resources and delete unused assets

10. Day-by-Day Implementation Plan

Day 1 - Requirements, Setup, and Profiling

1. Create a new Git repository and copy the supplied project structure.
2. Create a Python virtual environment and install requirements.txt.
3. Create PostgreSQL database retailmart and four schemas: staging, core, mart, audit.
4. Profile every supplied file: row count, columns, inferred types, nulls, duplicates, min/max dates, distinct categories, and invalid examples.
5. Create docs/data_profile.md and docs/data_dictionary.md.
6. Create an ER diagram and architecture diagram.
7. Do not clean or load data on Day 1.

Required evidence: source code/SQL, terminal output or screenshot, audit query results, and a short explanation of decisions.

Day 2 - PostgreSQL Design

8. Complete SQL DDL for staging/core/mart/audit tables.
9. Choose primary keys, foreign keys, NOT NULL, CHECK, and UNIQUE constraints.
10. Create indexes for frequent filters and joins; explain every index.
11. Create audit.pipeline_run and audit.file_audit tables.
12. Run DDL from a clean database twice to prove scripts are rerunnable.

Required evidence: source code/SQL, terminal output or screenshot, audit query results, and a short explanation of decisions.

Day 3 - Python Extraction and Validation

13. Implement configuration and structured logging.
14. Implement CSV/JSON readers that add source_file, ingestion_ts, and run_id.
15. Implement schema validation before row validation.
16. Implement DQ rules as reusable functions returning accepted and rejected DataFrames.
17. Write rejected files with rejection_reason and rejection_rule_ids.
18. Write pytest tests for at least eight rules.

Required evidence: source code/SQL, terminal output or screenshot, audit query results, and a short explanation of decisions.

Day 4 - PostgreSQL Loading and Transformations

19. Load staging tables in batches.
20. Upsert accepted master data to core tables.
21. Load sales incrementally with sale_id uniqueness.
22. Calculate monetary columns using Decimal-safe or explicitly rounded logic.
23. Create mart daily, product, store, and customer summaries.
24. Reconcile input, accepted, rejected, and database counts.

Required evidence: source code/SQL, terminal output or screenshot, audit query results, and a short explanation of decisions.

Day 5 - GCS and BigQuery

25. Create one GCS bucket and the required folder layout.
26. Upload original raw files unchanged.
27. Write curated sales as Parquet if pyarrow works; otherwise use CSV and document the choice.
28. Create BigQuery dataset retailmart_dw.
29. Create/load dimensions and fact_sales.
30. Partition fact_sales by business_date and cluster by store_id, product_id.
31. Run BigQuery reconciliation and document costs/bytes processed.

Required evidence: source code/SQL, terminal output or screenshot, audit query results, and a short explanation of decisions.

Day 6 - Analytics, Reliability, and Portfolio

32. Answer all twenty business questions.
33. Test a rerun of one daily file and prove no duplicates.
34. Simulate one malformed file and document failure behavior.
35. Complete README with architecture, commands, evidence, decisions, limitations, and screenshots.
36. Create a two-minute interview explanation and five resume-ready project bullets.
37. Tag the repository v1.0 only after acceptance criteria pass.

Required evidence: source code/SQL, terminal output or screenshot, audit query results, and a short explanation of decisions.

11. Exact Deliverables

Deliverable	Required File
Business understanding	docs/business_notes.md
Data profiling	docs/data_profile.md
Data dictionary	docs/data_dictionary.md
Architecture	docs/architecture.png
ER diagram	docs/er_diagram.png
DDL	sql/01_create_schema.sql plus table scripts
DQ SQL	sql/02_quality_queries.sql
ETL modules	src/*.py
Tests	tests/test_validation.py and others
Rejected outputs	data/rejected/...
GCS/BQ evidence	screenshots and README
Analytics	sql/03_analytics_questions.sql
Runbook	docs/runbook.md
Final portfolio	README.md

12. Acceptance Criteria

38. All mandatory files are processed or a clear error is produced.
39. Source count = accepted count + rejected count for each file.
40. All DQ rules have code and evidence.
41. Rejected rows retain original values plus reason codes.
42. No duplicate sale_id exists in PostgreSQL or BigQuery.
43. A repeated run produces zero additional sales rows.
44. All accepted foreign keys resolve to accepted dimensions.
45. PostgreSQL and BigQuery accepted sales counts match.
46. fact_sales is partitioned and clustered as required.
47. At least eight automated tests pass.
48. README allows another learner to reproduce the project.
49. No password, key file, or private data is committed.

13. Twenty Business Questions

50. Daily gross and net revenue
51. Revenue by store and region
52. Top 10 products by net revenue
53. Top categories by margin
54. Average order value by payment method
55. Discount impact on net revenue
56. Daily transaction count and units sold
57. Repeat customers during the seven-day period
58. Top customers by net revenue
59. Products sold in the most stores
60. Stores with high discount percentage
61. Sales during holidays versus non-holidays
62. Inventory below reorder level
63. Products with inventory but no sales
64. Sell-through proxy by store and product
65. Revenue contribution percentage by category
66. Running daily revenue
67. Three-day moving average of revenue
68. Rank stores within each region
69. Detect suspicious transactions with excessive discount or unusual quantity

14. Resources

Use official documentation first. These are learning resources, not proof of project completion.

Topic	Resource
Python	https://docs.python.org/3/tutorial/
pandas	https://pandas.pydata.org/docs/getting_started/intro_tutorials/
PostgreSQL	https://www.postgresql.org/docs/current/
SQLAlchemy	https://docs.sqlalchemy.org/
pytest	https://docs.pytest.org/
GCS Python	https://cloud.google.com/storage/docs/reference/libraries
BigQuery Python	https://cloud.google.com/bigquery/docs/python-libraries
BigQuery partitioning	https://cloud.google.com/bigquery/docs/partitioned-tables
BigQuery clustering	https://cloud.google.com/bigquery/docs/clustered-tables
Git	https://git-scm.com/docs/gittutorial

15. Review Questions - Answer Before Calling the Project Complete

70. Why use PostgreSQL before BigQuery in this design?
71. What makes your sales load idempotent?
72. How do you distinguish file-level failure from row-level rejection?
73. Why keep raw files unchanged?
74. How are accepted and rejected counts reconciled?
75. Which indexes did you create and what query benefits?
76. Why partition by business_date?
77. Why cluster by store_id and product_id?
78. How would you process 100 GB instead of 20,000 rows?
79. What would Airflow, Spark, and Kafka add in later versions?

16. Resume and Interview Rules

Only add this project to your resume after you complete it and can explain the code. Label it as a Personal Project or Portfolio Project. Do not say that RetailMart is a real employer or client.

Suggested honest title after completion: RetailMart Enterprise Sales Data Platform - Personal Data Engineering Project.

17. First Action

Start with Day 1 only. Do not request solution code before submitting your profiling, proposed schema, keys, validation rules, ER diagram, and architecture. The purpose is to learn engineering decisions, not merely execute generated code.